Sanbook Pay

White-Label UPI Payment Gateway - Step-by-Step Manual

Document	Step-by-Step Manual - Merchant Creation to Intent Return
Version	1.0
Date	November 2026
Platform	Sanbook Pay (youpi-pg)
Payment Mode	UPI Collection (No KYC, No Bank, No Fund Holding)
Flow Covered	Create Merchant → Setup UPI VPA → Create Payment Link → Customer Pays → Intent

This manual walks through the complete UPI payment lifecycle from merchant onboarding to payment intent return and ledger verification. Every confirmed payment posts atomically to a double-entry ledger.

Table of Contents

- 1. Phase 1 - Platform Owner Onboards Merchant
- 2. Phase 2 - Merchant Sets Up UPI Payment Receiving
- 3. Phase 3 - Merchant Creates a Payment Order (UPI QR)
- 4. Phase 4 - Customer Pays & Real-Time Callback
- 5. Phase 5 - Merchant Checks Payment Status & Intent Return
- 6. Phase 6 - Ledger Verification (Every Payment Recorded)
- 7. Android APK Integration (Mobile UPI)
- 8. API Reference (cURL Examples)
- 9. Troubleshooting

Architecture Overview

Sanbook Pay is a white-label UPI payment collection engine. The Platform Owner operates the central system and creates independent merchant accounts. Merchants receive UPI payments directly into their own VPA - the platform never holds funds.

Phase 1 - Platform Owner Onboards Merchant

The Platform Owner (Root Super Admin) creates the merchant account, sends a one-time invitation, and activates the merchant after approval. No KYC is required for UPI collection.

Step 1.1 - Create Merchant Account

```
POST /api/platform/v1/merchants
Headers: Authorization: Bearer <platform-admin-token>, X-CSRF-Token: <csrf>
{
"email": "owner@gamecompany.com",
"password": "OwnerPass12345!",
"businessName": "My Game Studio",
"contactName": "John Owner",
"mobile": "+919876543210",
"category": "gaming",
"address": "123 Game Street, Mumbai",
"registrationNumber": "GST123456789",
"taxId": "PANABCDE1234F"
}
```

Response includes **merchantId** (e.g. *m_abc123*) and *status: EMAIL_PENDING*.

Step 1.2 - Send Owner Invitation

```
POST /api/platform/v1/merchants/m_abc123/invite-owner
Headers: Authorization: Bearer <platform-admin-token>
```

The owner receives an email + SMS with a one-time setup link (expires in 24 hours, single-use).

Step 1.3 - Merchant Accepts Invitation & Sets Password

```
POST /api/platform/v1/owner-invitations/accept
{
"token": "<one-time-token>",
"password": "OwnerPass12345!"
}
```

Step 1.4 - Merchant Login (MFA Required)

```
POST /api/platform/v1/login
{ "email": "owner@gamecompany.com", "password": "OwnerPass12345!", "merchantId": "m_abc123"
}
Then: POST /api/platform/v1/login/mfa { "challenge": "<token>", "code": "123456" }
```

Session cookie *dp_platform_session* is set (HttpOnly, SameSite=Strict). CSRF token returned for state-changing requests.

Step 1.5 - Complete Business Onboarding

```
POST /api/platform/v1/onboarding
Headers: Cookie: dp_platform_session=<session>, X-CSRF-Token: <csrf>
{
"businessName": "My Game Studio",
"contactName": "John Owner",
"mobile": "+919876543210",
"category": "gaming",
"address": "123 Game Street, Mumbai",
"registrationNumber": "GST123456789",
"taxId": "PANABCDE1234F",
"submit": true
}
```

Status: *UNDER_REVIEW*. Platform owner must approve in Step 1.6.

Step 1.6 - Platform Owner Activates Merchant

```
POST /api/platform/v1/merchants/m_abc123/state
{ "status": "ACTIVE", "reason": "Merchant verified and approved for gaming payments" }
```

Merchant is now **ACTIVE** and can create payment links.

Phase 2 - Merchant Sets Up UPI Payment Receiving

The merchant registers their UPI VPA (Virtual Payment Address) and callback URL. Money goes directly to the merchant VPA - platform holds nothing.

Step 2.1 - Register UPI VPA

```
POST /api/v1/upi/merchant/setup

Headers: Cookie: dp_platform_session=<session>

{

"merchantId": "m_abc123",

"upiVpa": "mygame@upi",

"callbackUrl": "https://gamebackend.example.com/upi-callback",

"businessName": "My Game Studio",

"contactEmail": "owner@gamecompany.com",

"contactPhone": "+919876543210"

}
```

Response - everything the merchant receives:

```
{

"merchantId": "m_abc123",

"businessName": "My Game Studio",

"upiVpa": "mygame@upi",

"callbackUrl": "https://gamebackend.example.com/upi-callback",

"webhookSecret": "<32-char-base64>",

"accessToken": "<32-char-base64>",

"accessLink": "/merchant/m_abc123",

"apiBaseUrl": "https://your-domain.com"

}
```

Merchant Credentials

Share your UPI VPA (mygame@upi) with customers. They pay directly to this VPA. Save the Webhook Secret - you will verify HMAC-SHA256 signatures on incoming callbacks.

Step 2.2 - Multi-Operator Staff Setup (Optional)

The merchant can invite staff with roles: OWNER, ADMIN, DEVELOPER, FINANCE, SUPPORT, VIEWER.

```
POST /api/v1/multi-operator/users

{ "merchantId": "m_abc123", "email": "finance@gamecompany.com", "roleId": "finance-role",
"invitedBy": "owner" }
```

Phase 3 - Merchant Creates a Payment Order (UPI QR)

The merchant creates an amount-only payment link. The system generates a UPI QR URI and checkout page. No manual Order ID, customer name, mobile, or UTR required.

Step 3.1 - Create Payment Order

```
POST /api/v1/upi/orders
Headers: X-Merchant-Id: m_abc123
{
"orderId": "GAME-ORDER-001",
"amount": "100.00",
"currency": "INR",
"callbackUrl": "https://gamebackend.example.com/upi-callback"
}
```

Response:

```
{
"id": "<order-uuid>",
"orderId": "GAME-ORDER-001",
"amountPaise": 10000,
"currency": "INR",
"status": "PENDING",
"upiVpa": "mygame@upi",
"qrUri": "upi://pay?pa=mygame%40upi&pn;=Merchant&mode;=04&tr;=GAME-ORDER-001&tn;=Payment+GAM
E-ORDER-001&am;=100.00&cu;=INR",
"callbackUrl": "https://gamebackend.example.com/upi-callback"
}
```

The merchant displays the QR URI (or QR code image) to the customer. Customer pays ■100.00 to mygame@upi from any UPI app (Google Pay, PhonePe, Paytm, BHIM, Amazon Pay, CRED).

Step 3.2 - Customer Checkout Page

The merchant shares the checkout link: *https://your-domain.com/checkout/<payment-id>*. The page shows:

• Payable amount (■100.00)

• Live UPI QR code (auto-generated SVG)

• Payee VPA (mygame@upi) with copy-to-clipboard

• App launcher grid: Google Pay, PhonePe, Paytm, Generic UPI

• Sticky mobile "I've Paid" button

• Auto-polling payment status every 3 seconds

Phase 4 - Customer Pays & Real-Time Callback

Customer opens any UPI app, scans the QR, and pays ■100.00 to mygame@upi. Money goes directly to the merchant bank account.

Step 4.1 - Customer Scans and Pays

Customer opens Google Pay / PhonePe / Paytm / BHIM, scans the QR, confirms payment of ■100.00. Funds are transferred to the merchant VPA immediately.

Step 4.2 - Observer Detects Credit & Posts UTR

The Android observer app (or SMS/Gmail monitor) detects the UPI credit notification and posts:

```
POST /api/v1/upi/notify
Headers: X-Merchant-Id: m_abc123, X-Merchant-Secret: <webhook-secret>
{
"orderId": "GAME-ORDER-001",
"amountPaise": 10000,
"utr": "123456789012",
"senderVpa": "customer@upi",
"source": "ANDROID_OBSERVER",
"rawText": "■100.00 credited to mygame@upi from customer@upi. UTR: 123456789012"
}
```

Response:

```
{
"matched": true,
"orderId": "GAME-ORDER-001",
"status": "SUCCESS",
"utr": "123456789012",
"callback": { "sent": true, "status": 200 }
}
```

Step 4.3 - Game Backend Receives Real-Time Callback

```
POST https://gamebackend.example.com/upi-callback
Headers:
Content-Type: application/json
X-Gateway-Event: PAYMENT_SUCCESS
X-Gateway-Signature: sha256=<hmac-of-payload-using-webhook-secret>
X-Gateway-Delivery-Id: <order-uuid>
X-Gateway-Timestamp: <timestamp>
{
"event": "PAYMENT_SUCCESS",
```

```
"orderId": "GAME-ORDER-001",
"paymentId": "<order-uuid>",
"amount": "100.00",
"amountPaise": 10000,
"currency": "INR",
"status": "SUCCESS",
"utr": "123456789012",
"senderVpa": "customer@upi",
"payeeVpa": "mygame@upi",
"completedAt": "2026-11-01T08:46:00.000Z"
}
```

Game backend verifies: 1) HMAC-SHA256 signature = authentic, 2) orderId matches expected order, 3) amountPaise matches expected amount, 4) status is SUCCESS, 5) UTR is the 12-digit reference. Then credits the player wallet.

Phase 5 - Merchant Checks Payment Status & Intent Return

The merchant queries order status to confirm payment. The customer sees intent-return status on the checkout page.

Step 5.1 - Merchant Queries Order Status

```
GET /api/v1/upi/orders/GAME-ORDER-001
Headers: X-Merchant-Id: m_abc123

{
"id": "<order-uuid>",
"orderId": "GAME-ORDER-001",
"amountPaise": 10000,
"currency": "INR",
"status": "SUCCESS",
"upiVpa": "mygame@upi",
"utr": "123456789012",
"senderVpa": "customer@upi",
"createdAt": "...",
"completedAt": "..."
}
```

Step 5.2 - Customer Checkout Intent Return

The customer checkout page auto-polls every 3 seconds. Status transitions:

• **PENDING** → Waiting for customer to pay

• **INTENT_OPENED** → UPI app opened (Android deep-link)

• **CLIENT_REPORTED_SUCCESS** → Customer clicked "I've Paid"

• **SUCCESS** → Payment confirmed by bank observers / UTR

• **FAILED / EXPIRED / CANCELLED** → Payment did not complete

On SUCCESS, the customer sees "Payment Successful!" and can return to the merchant store via the return URL.

Step 5.3 - Payment-Done Verification Request

```
POST /api/v1/checkout/<payment-id>/payment-done
Headers: Content-Type: application/json
Body: {}
```

Customer clicks "I've Paid" to request verified confirmation. Status updates automatically as bank observers confirm.

Phase 6 - Ledger Verification (Every Payment Recorded)

Every confirmed UPI payment posts atomically to a double-entry financial ledger. Integer-paise accounting, no floating-point rounding.

Step 6.1 - Ledger Entry on Confirmation

```
Debit: PROVIDER_CLEARING (Asset) → 10000 paise
Credit: MERCHANT_RECEIVABLES (Asset) → 10000 paise
Reference Type: COLLECTION | Reference ID: <order-uuid>
Narration: Provider confirmed collection. Bank reference: 123456789012
Status: POSTED (atomic - both entries commit or none)
```

Step 6.2 - Query Merchant Journal

```
GET /api/v1/ledger/journal?merchantId=m_abc123&limit;=50
```

Returns all posted transactions with debit/credit entries, account codes, and amounts.

Step 6.3 - Trial Balance

```
GET /api/v1/ledger/trial-balance?merchantId=m_abc123
```

Returns all ledger account balances: MERCHANT_RECEIVABLES, PROVIDER_CLEARING, BANK_CASH, FEES, REFUND_LIABILITIES, etc.

Ledger Properties

• Atomic posting - both debit and credit commit together • Idempotent - same reference cannot post twice • Immutable - historical entries never silently rewritten • Integer paise - no floating-point errors

Android APK Integration (Mobile UPI)

The Android Bridge APK enables mobile-native UPI intent launching and notification observation.

APK Path: *apps/android-bridge/app/build/outputs/apk/debug/app-debug.apk* (7.5 MB)

Build: *cd apps/android-bridge && ./gradlew assembleDebug* (Android SDK 34, Java 17)

Install: *adb install app-debug.apk*

Android UPI App Launcher Tiles:

App	Package Name	Deep Link Scheme
Google Pay	com.google.android.apps.nbu.paisa.user	upi://pay?...#Intent;scheme=upi;package=...;end
PhonePe	com.phonepe.app	upi://pay?...#Intent;scheme=upi;package=...;end
Paytm	net.one97.paytm	upi://pay?...#Intent;scheme=upi;package=...;end
Generic UPI	(any)	upi://pay?pa=...&am=...&tr=...&cu=INR

Android Flow

Customer taps a UPI app tile → app opens with pre-filled payment → customer pays → returns to checkout page → status updates automatically.

API Reference - cURL Examples

1. Register Merchant UPI VPA

```
curl -X POST http://localhost:6100/api/v1/upi/merchant/setup \
-H "Content-Type: application/json" \
-d '{"merchantId":"m_demo","upiVpa":"mygame@upi","callbackUrl":"https://game.example.com/callback","businessName":"Demo Game"}'
```

2. Create Payment Order

```
curl -X POST http://localhost:6100/api/v1/upi/orders \
-H "Content-Type: application/json" \
-H "X-Merchant-Id: m_demo" \
-d '{"orderId":"GAME-001","amount":"100.00"}'
```

3. Simulate Payment Received (Observer Posts UTR)

```
curl -X POST http://localhost:6100/api/v1/upi/notify \
-H "Content-Type: application/json" \
-H "X-Merchant-Id: m_demo" \
-H "X-Merchant-Secret: <webhook-secret>" \
-d '{"orderId":"GAME-001","amountPaise":10000,"utr":"123456789012","senderVpa":"customer@upi","source":"ANDROID_OBSERVER"}'
```

4. Check Order Status

```
curl http://localhost:6100/api/v1/upi/orders/GAME-001 \
-H "X-Merchant-Id: m_demo"
```

5. Request Payment-Done Verification

```
curl -X POST http://localhost:6100/api/v1/checkout/<payment-id>/payment-done \
-H "Content-Type: application/json" \
-d '{}'
```

Troubleshooting

Issue	Cause	Fix
Payment not detected	Observer not running or wrong VPA	Start Android observer/SMS/Gmail monitor; verify payeeVpa ma
Callback not received	HMAC signature mismatch or endpoint down	Verify webhookSecret; ensure callbackUrl is public and returns 2
Ledger entry missing	Payment not confirmed	Wait for provider confirmation; check order status is SUCCESS
QR not loading	Session expired or inactive	Create new payment order; refresh checkout page
MFA required	Sensitive operation without authentication	Complete login + MFA flow first
Circuit breaker open	Provider connection failures	Wait for recovery timeout or manually reset via circuit-breaker co